1. Bank Loan Analysis Report

1.0.1 Importing Liabraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import plotly.express as px
```

1.0.2 Import the Data File

```
df = pd.read_excel('financial_loan.xlsx')
```

```
df.head()
```

Columns 1–8 of 22

	id	address_state	application_type	emp_length	emp_title	grade	home_ownership
0	1077430	GA	INDIVIDUAL	< 1 year	Ryder	C	RENT
1	1072053	CA	INDIVIDUAL	9 years	MKC Accounting	E	RENT
2	1069243	CA	INDIVIDUAL	4 years	Chemat Technology Inc	C	RENT
3	1041756	TX	INDIVIDUAL	< 1 year	barnes distribution	B	MORTGAGE
4	1068350	IL	INDIVIDUAL	10+ years	J&J Steel Inc	A	MORTGAGE

Columns 9–15 of 22 · first column repeated

	issue_date	last_credit_pull_date	last_payment_date	...	sub_grade	term	verification_status
0	2021-02-11	2021-09-13	2021-04-13	...	C4	60 months	Source Verified
1	2021-01-01	2021-12-14	2021-01-15	...	E1	36 months	Source Verified
2	2021-01-05	2021-12-12	2021-01-09	...	C5	36 months	Not Verified
3	2021-02-25	2021-12-12	2021-03-12	...	B2	60 months	Source Verified
4	2021-01-01	2021-12-14	2021-01-15	...	A1	36 months	Verified

Columns 16–22 of 22 · first column repeated

	annual_income	dti	installment	int_rate	loan_amount	total_acc	total_payment
0	30000.0	0.0100	59.83	0.1527	2500	4	1009
1	48000.0	0.0535	109.43	0.1864	3000	4	3939
2	50000.0	0.2088	421.65	0.1596	12000	11	3522
3	42000.0	0.0540	97.06	0.1065	4500	9	4911
4	83000.0	0.0231	106.53	0.0603	3500	28	3835

5 rows × 24 columns

```
df.tail()
```

Columns 1–8 of 22

	id	address_state	application_type	emp_length	emp_title	grade	home_ownership
38571	803452	NJ	INDIVIDUAL	< 1 year	Joseph M Sanzari Company	C	MORTGAGE
38572	970377	NY	INDIVIDUAL	8 years	Swat Fame	C	RENT
38573	875376	CA	INDIVIDUAL	5 years	Anaheim Regional Medical Center	D	RENT
38574	972997	NY	INDIVIDUAL	5 years	Brooklyn Radiology	D	RENT
38575	682952	NY	INDIVIDUAL	4 years	Allen Edmonds	F	RENT

Columns 9–15 of 22 · first column repeated

	issue_date	last_credit_pull_date	last_payment_date	...	sub_grade	term	verification_status
38571	2021-07-11	2021-05-16	2021-05-16	...	C1	60 months	Verified
38572	2021-10-11	2021-04-16	2021-05-16	...	C1	60 months	Verified
38573	2021-09-11	2021-05-16	2021-05-16	...	D5	60 months	Verified
38574	2021-10-11	2021-05-16	2021-05-16	...	D5	60 months	Verified
38575	2021-07-11	2021-05-16	2021-05-16	...	F3	60 months	Verified

Columns 16–22 of 22 · first column repeated

	annual_income	dti	installment	int_rate	loan_amount	total_acc	total_payment
38571	100000.0	0.1986	551.64	0.1299	24250	33	31946
38572	50000.0	0.0458	579.72	0.1349	25200	18	31870
38573	65000.0	0.1734	627.93	0.1749	25000	20	35721
38574	368000.0	0.0009	612.72	0.1825	24000	9	33677
38575	80000.0	0.0600	486.86	0.2099	18000	7	27679

5 rows × 24 columns

1.0.3 Metadata of Data

```
print("No. of rows:", df.shape[0])
```

```
No. of rows: 38576
```

```
print("No. of columns:", df.shape[1])
```

```
No. of columns: 24
```

```
df.info
```

```
<bound method DataFrame.info of             id address_state application_type
emp_length  \
0      1077430            GA       INDIVIDUAL   < 1 year
1      1072053            CA       INDIVIDUAL    9 years
2      1069243            CA       INDIVIDUAL    4 years
3      1041756            TX       INDIVIDUAL   < 1 year
4      1068350            IL       INDIVIDUAL  10+ years
...        ...           ...              ...        ...
38571   803452            NJ       INDIVIDUAL   < 1 year
38572   970377            NY       INDIVIDUAL    8 years
38573   875376            CA       INDIVIDUAL    5 years
38574   972997            NY       INDIVIDUAL    5 years
38575   682952            NY       INDIVIDUAL    4 years

                           emp_title grade home_ownership issue_date  \
0                              Ryder     C           RENT 2021-02-11
1                        MKC Accounting     E           RENT 2021-01-01
2                  Chemat Technology Inc     C           RENT 2021-01-05
3                    barnes distribution     B       MORTGAGE 2021-02-25
4                          J&J Steel Inc     A       MORTGAGE 2021-01-01
...                                  ...   ...            ...        ...
38571         Joseph M Sanzari Company     C       MORTGAGE 2021-07-11
38572                          Swat Fame     C           RENT 2021-10-11
38573  Anaheim Regional Medical Center     D           RENT 2021-09-11
38574                Brooklyn Radiology     D           RENT 2021-10-11
38575                     Allen Edmonds     F           RENT 2021-07-11

      last_credit_pull_date last_payment_date  ... sub_grade        term  \
0                2021-09-13        2021-04-13  ...        C4   60 months
1                2021-12-14        2021-01-15  ...        E1   36 months
2                2021-12-12        2021-01-09  ...        C5   36 months
3                2021-12-12        2021-03-12  ...        B2   60 months
4                2021-12-14        2021-01-15  ...        A1   36 months
...                     ...               ...  ...       ...         ...
38571            2021-05-16        2021-05-16  ...        C1   60 months
38572            2021-04-16        2021-05-16  ...        C1   60 months
38573            2021-05-16        2021-05-16  ...        D5   60 months
38574            2021-05-16        2021-05-16  ...        D5   60 months
38575            2021-05-16        2021-05-16  ...        F3   60 months

       verification_status annual_income     dti installment int_rate  \
0          Source Verified       30000.0  0.0100       59.83   0.1527
1          Source Verified       48000.0  0.0535      109.43   0.1864
2             Not Verified       50000.0  0.2088      421.65   0.1596
3          Source Verified       42000.0  0.0540       97.06   0.1065
4                 Verified       83000.0  0.0231      106.53   0.0603
...                    ...           ...     ...         ...      ...
38571             Verified      100000.0  0.1986      551.64   0.1299
38572             Verified       50000.0  0.0458      579.72   0.1349
38573             Verified       65000.0  0.1734      627.93   0.1749
38574             Verified      368000.0  0.0009      612.72   0.1825
38575             Verified       80000.0  0.0600      486.86   0.2099
```

```
       loan_amount  total_acc  total_payment
0             2500          4           1009
1             3000          4           3939
2            12000         11           3522
3             4500          9           4911
4             3500         28           3835
...            ...        ...            ...
38571        24250         33          31946
38572        25200         18          31870
38573        25000         20          35721
38574        24000          9          33677
38575        18000          7          27679

[38576 rows x 24 columns]>
```

1.0.4 Data Types

```
df.dtypes
```

```
id                              int64
address_state                     str
application_type                  str
emp_length                        str
emp_title                      object
grade                             str
home_ownership                    str
issue_date               datetime64[us]
last_credit_pull_date    datetime64[us]
last_payment_date        datetime64[us]
loan_status                       str
next_payment_date        datetime64[us]
member_id                        int64
purpose                           str
sub_grade                         str
term                              str
verification_status               str
annual_income                  float64
dti                            float64
installment                    float64
int_rate                       float64
loan_amount                      int64
total_acc                        int64
total_payment                    int64
dtype: object
```

```
df.describe()
```

Columns 1–8 of 14

	id	issue_date	last_credit_pull_date	last_payment_date	next_payment_date	member_id	annual_income
count	3.857600e+04	38576	38576	38576	38576	3.857600e+04	3.857600e+04
mean	6.810371e+05	2021-07-16 02:31:35.562007	2021-06-08 13:36:34.193280	2021-06-26 09:52:08.909166	2021-07-26 20:42:20.605557	8.476515e+05	6.964454e+04
min	5.473400e+04	2021-01-01 00:00:00	2021-01-08 00:00:00	2021-01-08 00:00:00	2021-02-08 00:00:00	7.069900e+04	4.000000e+03
25%	5.135170e+05	2021-04-11 00:00:00	2021-04-15 00:00:00	2021-03-16 00:00:00	2021-04-16 00:00:00	6.629788e+05	4.150000e+04
50%	6.627280e+05	2021-07-11 00:00:00	2021-05-16 00:00:00	2021-06-14 00:00:00	2021-07-14 00:00:00	8.473565e+05	6.000000e+04
75%	8.365060e+05	2021-10-11 00:00:00	2021-08-13 00:00:00	2021-09-15 00:00:00	2021-10-15 00:00:00	1.045652e+06	8.320050e+04
max	1.077501e+06	2021-12-12 00:00:00	2022-01-20 00:00:00	2021-12-15 00:00:00	2022-01-15 00:00:00	1.314167e+06	6.000000e+06
std	2.113246e+05	NaN	NaN	NaN	NaN	2.668105e+05	6.429368e+04

Columns 9–14 of 14 · first column repeated

	dti	installment	int_rate	loan_amount	total_acc	total_payment
count	38576.000000	38576.000000	38576.000000	38576.000000	38576.000000	38576.000000
mean	0.133274	326.862965	0.120488	11296.066855	22.132544	12263.348533
min	0.000000	15.690000	0.054200	500.000000	2.000000	34.000000
25%	0.082100	168.450000	0.093200	5500.000000	14.000000	5633.000000
50%	0.134200	283.045000	0.118600	10000.000000	20.000000	10042.000000
75%	0.185900	434.442500	0.145900	15000.000000	29.000000	16658.000000
max	0.299900	1305.190000	0.245900	35000.000000	90.000000	58564.000000
std	0.066662	209.092000	0.037164	7460.746022	11.392282	9051.104777

1.0.5 Total Loan Applications

```
total_loan_application = df['id'].count()
print("Total loan Applications:", total_loan_application)
```

```
Total loan Applications: 38576
```

1.0.6 MTD Total Loan Applications

```
latest_issue_date = df['issue_date'].max()
latest_year = latest_issue_date.year
latest_month = latest_issue_date.month
mtd_date = df[(df['issue_date'].dt.year == latest_year) & (df['issue_date'].dt.month
== latest_month)]
mtd_loan_applications = mtd_date['id'].count()
print(f"MTD Loan Applications(for{latest_issue_date.strftime('%B %Y')}) :
{mtd_loan_applications})")
```

```
MTD Loan Applications(forDecember 2021) : 4314)
```

1.0.7 Total Funded Amount

```
total_funded_amount = df['loan_amount'].sum()
total_funded_amount_millions = total_funded_amount / 1000000
print("Total Funded Amount: ${:,.2f}M".format(total_funded_amount_millions))
```

```
Total Funded Amount: $435.76M
```

1.0.8 MTD - Total Funded Amount

```
latest_issue_date = df['issue_date'].max()
latest_year = latest_issue_date.year
latest_month = latest_issue_date.month

mtd_data = df[(df['issue_date'].dt.year == latest_year) & (df['issue_date'].dt.month
== latest_month)]

mtd_total_funded_amount = mtd_data['loan_amount'].sum()
mtd_total_funded_amount_millions = mtd_total_funded_amount / 1000000

print("Total Funded Amount: ${:,.2f}M".format(mtd_total_funded_amount_millions))
```

```
Total Funded Amount: $53.98M
```

1.0.9 Total Amount Received

```
total_amount_received = df['total_payment'].sum()
total_amount_received_millions = total_funded_amount / 1000000
print("Total_Amount_Received: ${:,.2f}M".format(total_amount_received_millions))
```

```
Total_Amount_Received: $435.76M
```

1.0.10 MTD - Total Amount Received

```
latest_issue_date = df['issue_date'].max()
latest_year = latest_issue_date.year
latest_month = latest_issue_date.month

mtd_data = df[(df['issue_date'].dt.year == latest_year) & (df['issue_date'].dt.month
== latest_month)]

mtd_total_amount_received = mtd_data['total_payment'].sum()
mtd_total_amount_received_millions = mtd_total_amount_received / 1000000

print("Total_Amount_Received: ${:,.2f}M".format(mtd_total_amount_received_millions))
```

```
Total_Amount_Received: $58.07M
```

1.0.11 Average Interest Rate

```
average_interest_rate = df['int_rate'].mean()*100
print("Average_Interest_Rate: {:,.2f}%".format(average_interest_rate))
```

```
Average_Interest_Rate: 12.05%
```

1.0.12 Average Debt-to-Income Ratio (DTI)

```
average_debt_to_income_ratio = df['dti'].mean()*100
print("Average_Debt_to_Income_Ratio: {:,.2f}%".format(average_debt_to_income_ratio))
```

```
Average_Debt_to_Income_Ratio: 13.33%
```

1.0.13 Good Loan Matrics

```
good_loans = df[df['loan_status'].isin(["Fully Paid", "Current"])]  # Good Loans

total_loan_applications = df['id'].count()

good_loan_applications = good_loans['id'].count()
good_loan_funded_amount = good_loans['loan_amount'].sum()
good_loan_received = good_loans['total_payment'].sum()

good_loan_funded_amount_millions = good_loan_funded_amount / 1000000
good_loan_received_millions = good_loan_received / 1000000

good_loan_percentage = (good_loan_applications / total_loan_applications) *100

print("Good Loan Applications:", good_loan_applications)
print("Good Loan Funded Amount (in Millions):
${:,.2f}M".format(good_loan_funded_amount_millions))
print("Good Loan Total Received(in Millions):
${:,.2f}M".format(good_loan_received_millions))
print("Percentage of Good Loan Applications: {:,.2f}%".format(good_loan_percentage))
```

```
Good Loan Applications: 33243
Good Loan Funded Amount (in Millions): $370.22M
Good Loan Total Received(in Millions): $435.79M
Percentage of Good Loan Applications: 86.18%
```

1.0.14 Bad Loan Matrics

```
bad_loans = df[df['loan_status'].isin(["Charged Off"])] # Bad Loan

total_loan_applications = df['id'].count()

bad_loan_applications = bad_loans['id'].count()
bad_loan_funded_amount = bad_loans['loan_amount'].sum()
bad_loan_received = bad_loans['total_payment'].sum()

bad_loan_funded_amount_millions = bad_loan_funded_amount / 1000000
bad_loan_received_millions = bad_loan_received / 1000000

bad_loan_percentage = (bad_loan_applications / total_loan_applications) * 100

print("Bad Loan Applications:", bad_loan_applications)
print("Bad Loan Funded Amount (in Millions):
${:,.2f}M".format(bad_loan_funded_amount_millions))
print("Bad Loan Total Received(in Millions):
${:,.2f}M".format(bad_loan_received_millions))
print("Percentage of Bad Loan Applications: {:,.2f}%".format(bad_loan_percentage))
```

```
Bad Loan Applications: 5333
Bad Loan Funded Amount (in Millions): $65.53M
Bad Loan Total Received(in Millions): $37.28M
Percentage of Bad Loan Applications: 13.82%
```

1.0.15 Monthly Trends by Issue Date for Total Funded Amount

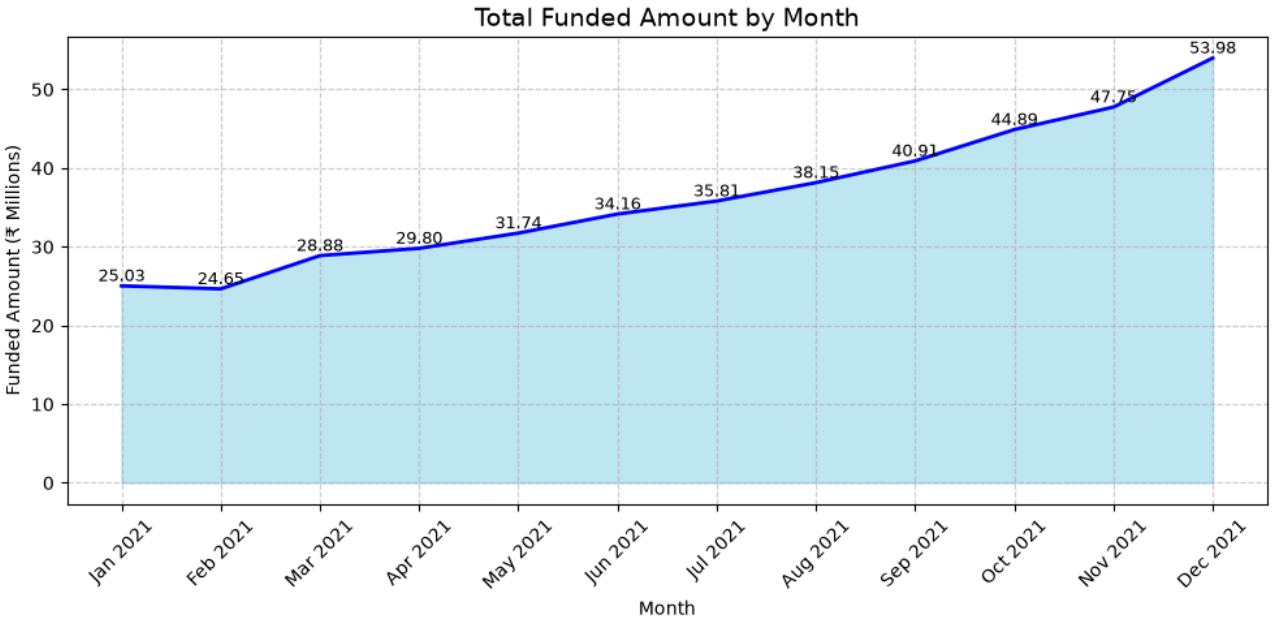
```
monthly_funded = (
    df.sort_values('issue_date')
      .assign(month_name = lambda x:x['issue_date'].dt.strftime('%b %Y'))
      .groupby('month_name', sort = False)['loan_amount']
      .sum()
      .div(1_000_000)
      . reset_index(name = 'loan_amount_millions')
)

plt.figure(figsize = (10,5))
plt.fill_between(monthly_funded['month_name'],
monthly_funded['loan_amount_millions'], color = 'skyblue', alpha = 0.5)
plt.plot(monthly_funded['month_name'], monthly_funded['loan_amount_millions'], color
= 'blue', linewidth = 2)

for i, row in monthly_funded.iterrows():

    plt.text(i, row['loan_amount_millions'] + 0.1, f"
{row['loan_amount_millions']:.2f}",
             ha = 'center', va = 'bottom', fontsize = 9, rotation = 0, color =
'black')


plt.title("Total Funded Amount by Month", fontsize = 14)
plt.xlabel("Month")
plt.ylabel("Funded Amount (₹ Millions)")
plt.xticks(ticks = range(len(monthly_funded)), label = monthly_funded['month_name'],
rotation = 45)
plt.grid(True, linestyle = "--", alpha = 0.6)
plt.tight_layout()
plt.show()
```



1.0.16 Monthly Trends by Issue Date for Total Amount Received

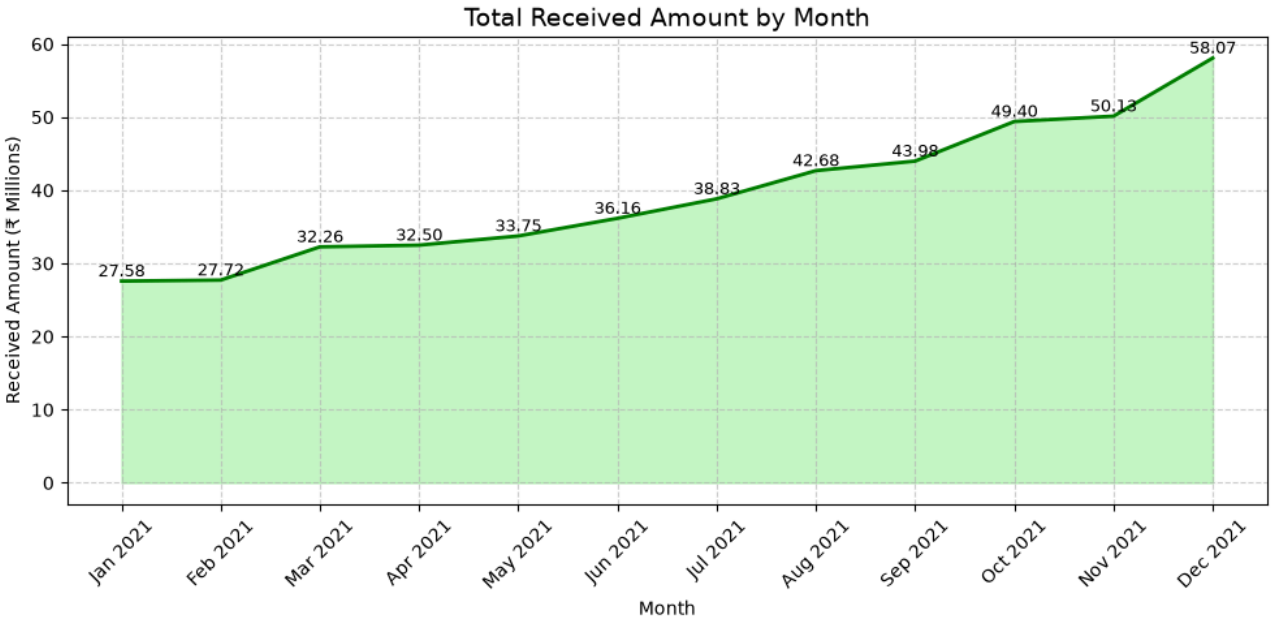
```
monthly_received = (
    df.sort_values('issue_date')
      .assign(month_name = lambda x:x['issue_date'].dt.strftime('%b %Y'))
      .groupby('month_name', sort = False)['total_payment']
      .sum()
      .div(1_000_000)
      . reset_index(name = 'received_amount_millions')
)

plt.figure(figsize = (10,5))
plt.fill_between(monthly_received['month_name'],
monthly_received['received_amount_millions'], color = 'lightgreen', alpha = 0.5)
plt.plot(monthly_received['month_name'],
monthly_received['received_amount_millions'], color = 'green', linewidth = 2)

for i, row in monthly_received.iterrows():

    plt.text(i, row['received_amount_millions'] + 0.1, f"
{row['received_amount_millions']:.2f}",
             ha = 'center', va = 'bottom', fontsize = 9, rotation = 0, color =
'black')


plt.title("Total Received Amount by Month", fontsize = 14)
plt.xlabel("Month")
plt.ylabel("Received Amount (₹ Millions)")
plt.xticks(ticks = range(len(monthly_received)), label =
monthly_received['month_name'], rotation = 45)
plt.grid(True, linestyle = "--", alpha = 0.6)
plt.tight_layout()
plt.show()
```



1.0.17 Monthly Trends by Issue Date for Total Loan Applications

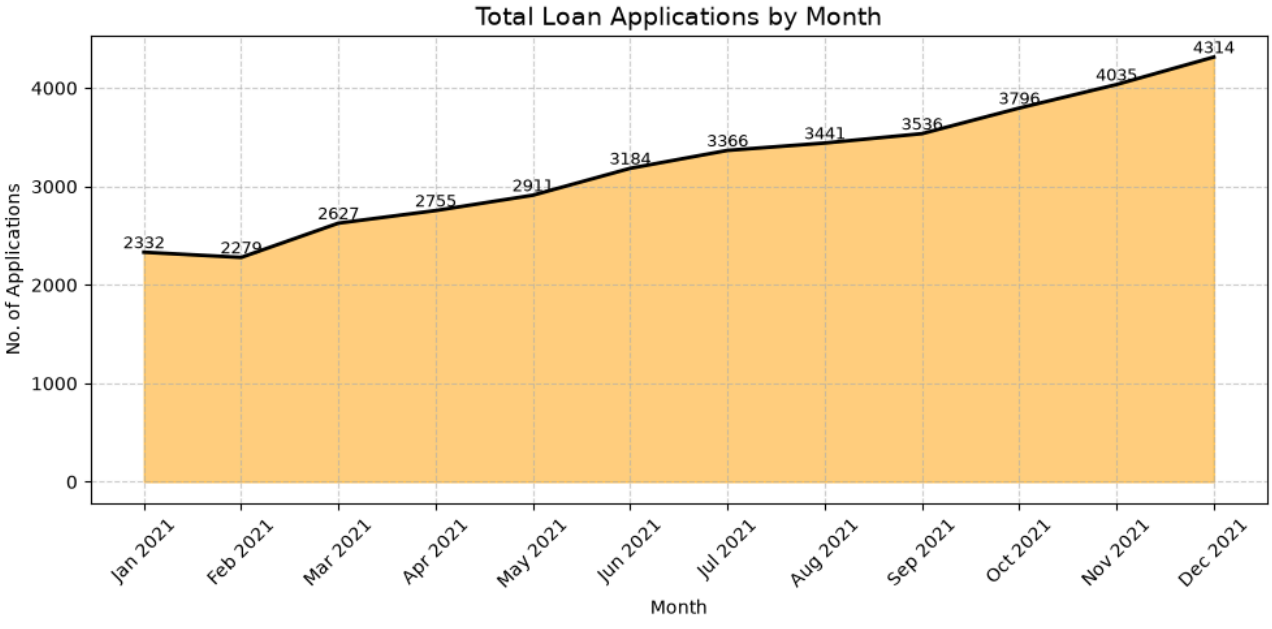
```
monthly_applications = (
    df.sort_values('issue_date')
      .assign(month_name = lambda x:x['issue_date'].dt.strftime('%b %Y'))
      .groupby('month_name', sort = False)['id']
      .count()
      . reset_index(name = 'loan_applications_count')
)

plt.figure(figsize = (10,5))
plt.fill_between(monthly_applications['month_name'],
monthly_applications['loan_applications_count'], color = 'orange', alpha = 0.5)
plt.plot(monthly_applications['month_name'],
monthly_applications['loan_applications_count'], color = 'black', linewidth = 2)

for i, row in monthly_applications.iterrows():

    plt.text(i, row['loan_applications_count'] + 0.1, f"
{row['loan_applications_count']:.0f}",
             ha = 'center', va = 'bottom', fontsize = 9, rotation = 0, color =
'black')


plt.title("Total Loan Applications by Month", fontsize = 14)
plt.xlabel("Month")
plt.ylabel("No. of Applications")
plt.xticks(ticks = range(len(monthly_applications)), label =
monthly_applications['month_name'], rotation = 45)
plt.grid(True, linestyle = "--", alpha = 0.6)
plt.tight_layout()
plt.show()
```



1.0.18 Regional Analysis for State for Total Funded Amount

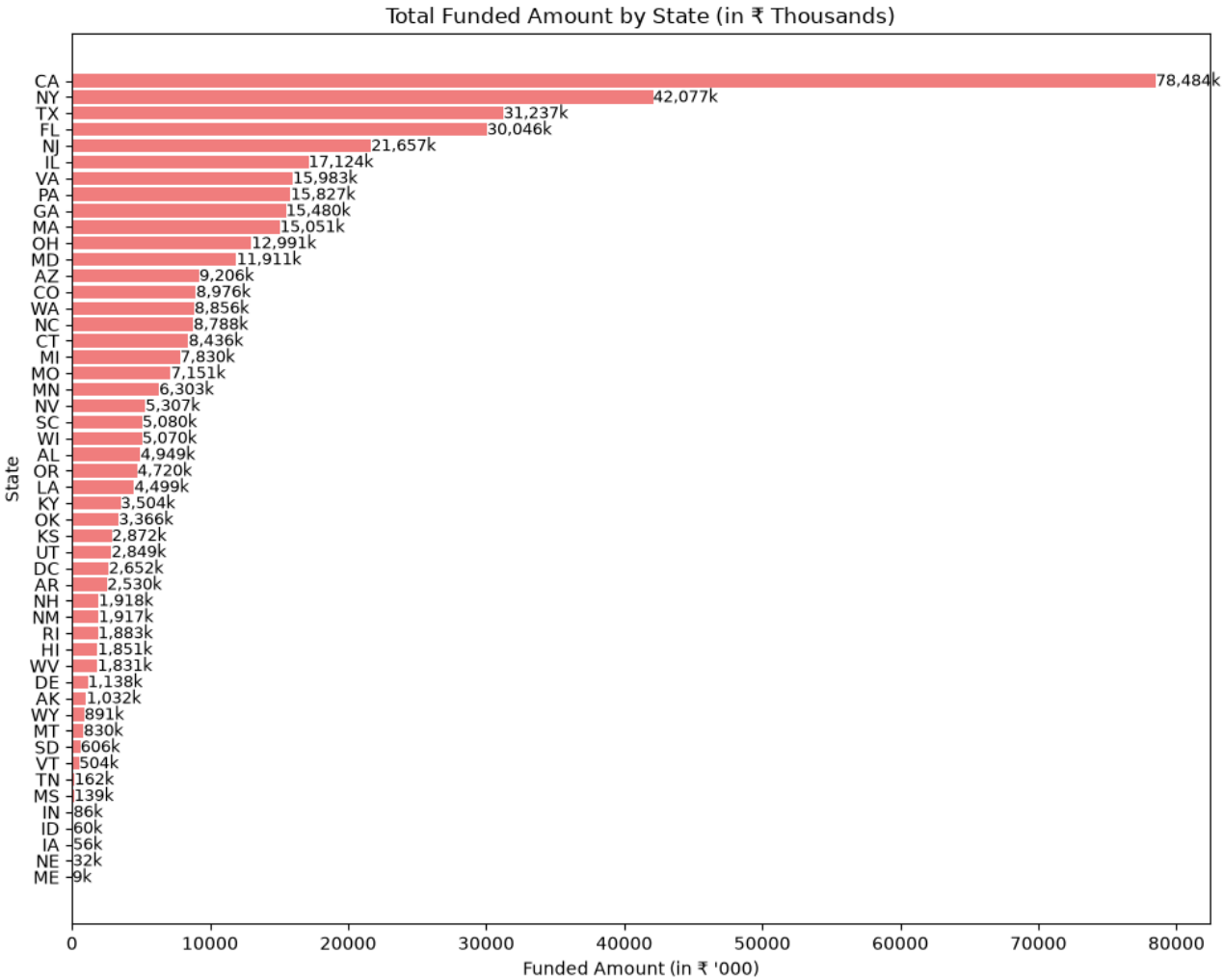
```
state_funding = df.groupby('address_state')
['loan_amount'].sum().sort_values(ascending = True)
state_funding_thousands = state_funding / 1000

plt.figure(figsize = (10,8))
bars = plt.barh(state_funding_thousands.index, state_funding_thousands.values, color
= 'lightcoral')

for bar in bars:
    width = bar.get_width()

    plt.text( width + 10, bar.get_y() + bar.get_height() / 2,
           f'{width:,.0f}k', va = 'center', fontsize = 9)

plt.title('Total Funded Amount by State (in ₹ Thousands)')
plt.xlabel('Funded Amount (in ₹ \'000)')
plt.ylabel('State')
plt.tight_layout()
plt.show()
```



1.0.19 Regional Analysis for State for Total Amount Received

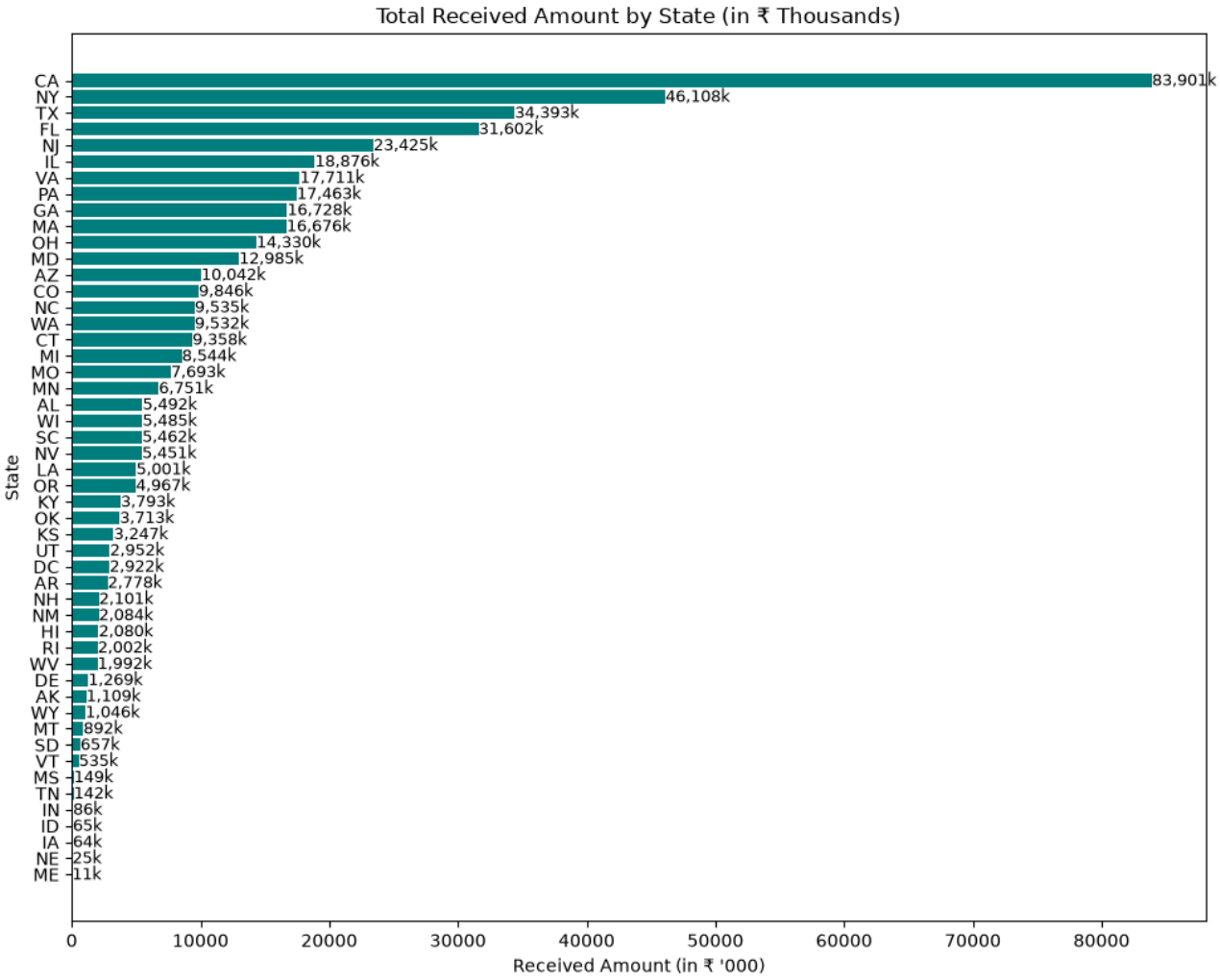
```
state_received = df.groupby('address_state')
['total_payment'].sum().sort_values(ascending = True)
state_received_thousands = state_received / 1000

plt.figure(figsize = (10,8))
bars = plt.barh(state_received_thousands.index, state_received_thousands.values,
color = 'teal')

for bar in bars:
    width = bar.get_width()

    plt.text( width + 10, bar.get_y() + bar.get_height() / 2,
           f'{width:,.0f}k', va = 'center', fontsize = 9)

plt.title('Total Received Amount by State (in ₹ Thousands)')
plt.xlabel('Received Amount (in ₹ \'000)')
plt.ylabel('State')
plt.tight_layout()
plt.show()
```



1.0.20 Regional Analysis for State for Total Loan Applications

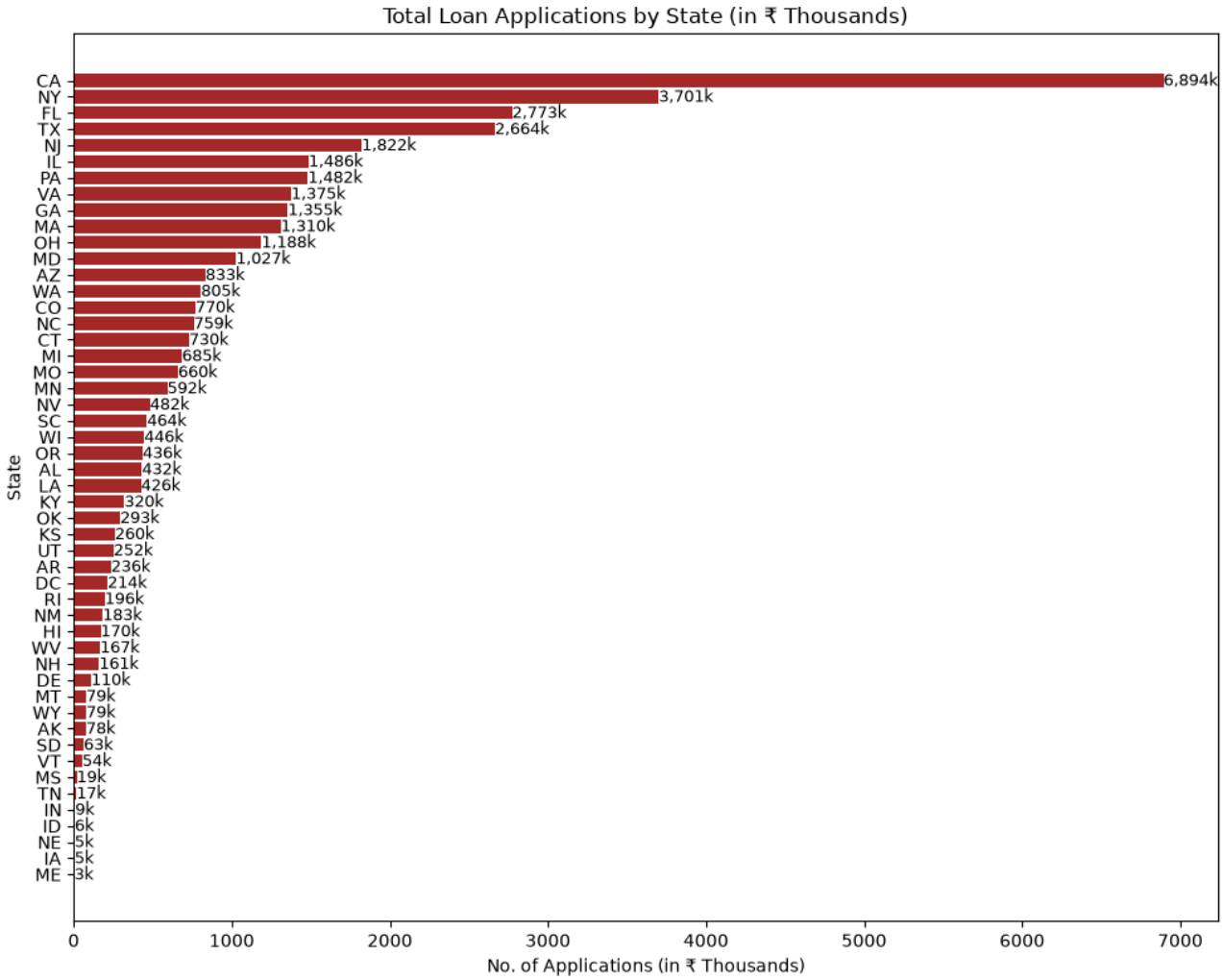
```
state_applications = df.groupby('address_state')['id'].count().sort_values(ascending
= True)

plt.figure(figsize = (10,8))
bars = plt.barh(state_applications.index, state_applications.values, color = 'brown')

for bar in bars:
    width = bar.get_width()

    plt.text( width + 0.05, bar.get_y() + bar.get_height() / 2,
           f'{width:,.0f}k', va = 'center', fontsize = 9)

plt.title('Total Loan Applications by State (in ₹ Thousands)')
plt.xlabel('No. of Applications (in ₹ Thousands)')
plt.ylabel('State')
plt.tight_layout()
plt.show()
```



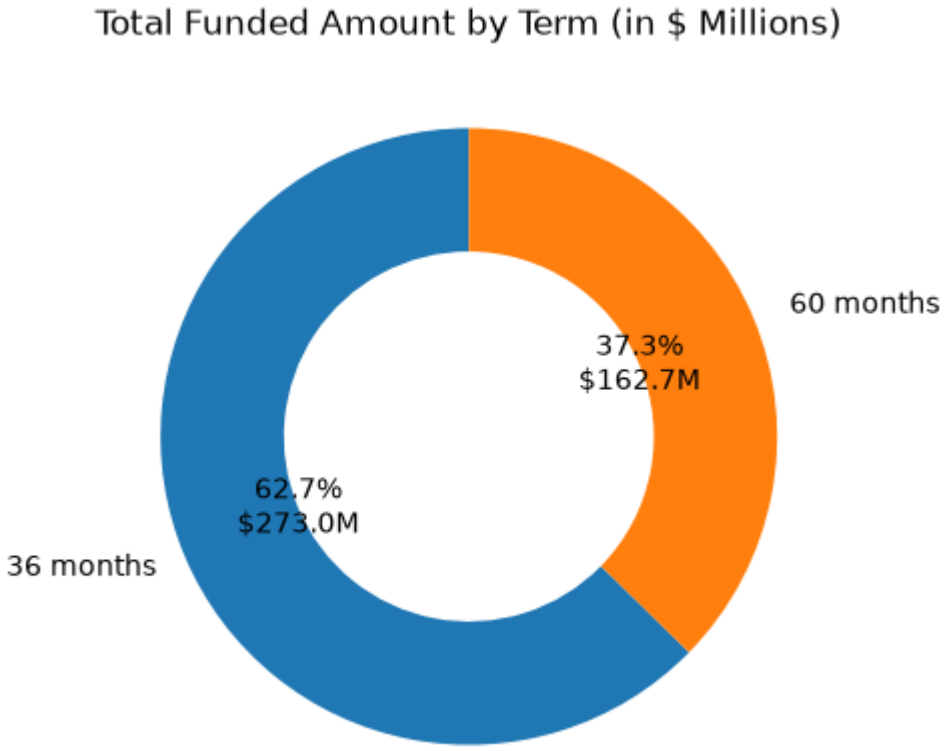
1.0.21 Long Term Analysis by Total Funded Amount

```
term_funding_millions =  df.groupby('term')['loan_amount'].sum() / 1000000

plt.figure(figsize=(5,5))

plt.pie(
    term_funding_millions,
    labels = term_funding_millions.index,
    autopct = lambda p: f"{p:.1f}%\n${p*sum(term_funding_millions)/100:.1f}M",
    startangle = 90,
    wedgeprops={'width':0.4}
)

plt.title('Total Funded Amount by Term (in $ Millions)')
plt.show()
```



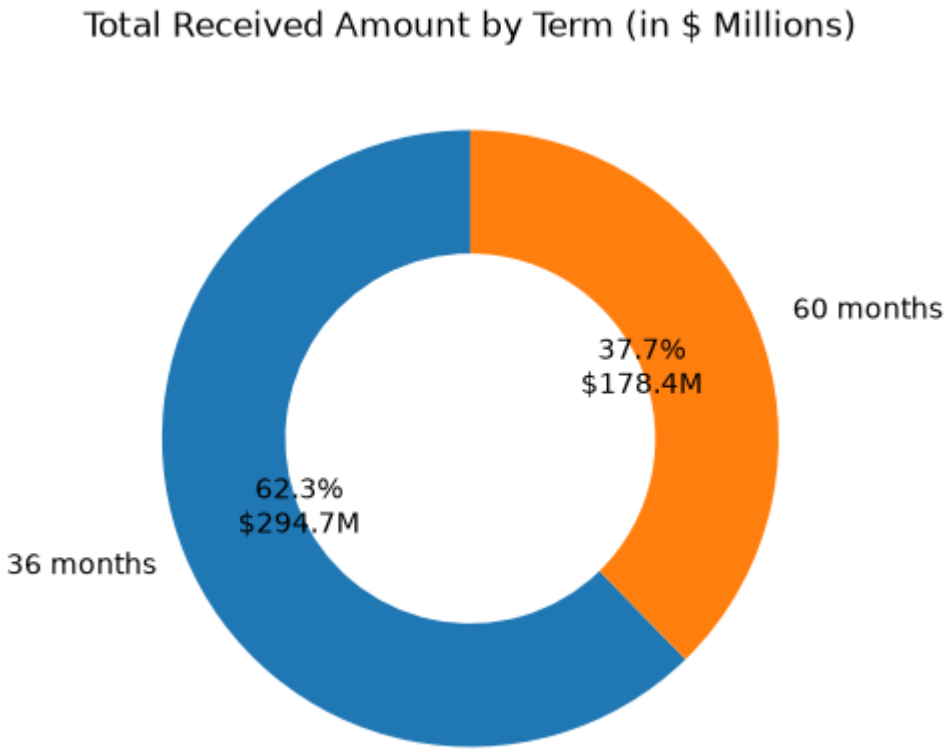
1.0.22 Long Term Analysis by Total Amount Received

```
term_received_millions =  df.groupby('term')['total_payment'].sum() / 1000000

plt.figure(figsize=(5,5))

plt.pie(
    term_received_millions,
    labels = term_received_millions.index,
    autopct = lambda p: f"{p:.1f}%\n${p*sum(term_received_millions)/100:.1f}M",
    startangle = 90,
    wedgeprops = {'width':0.4}
)

plt.title('Total Received Amount by Term (in $ Millions)')
plt.show()
```



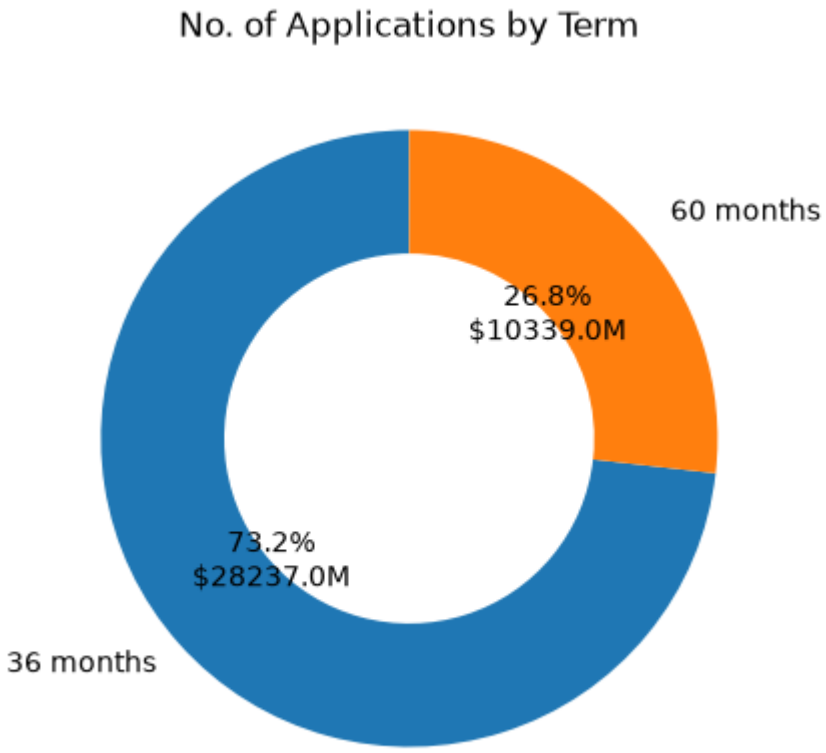
1.0.23 Long Term Analysis by Total Loan Applications

```
total_term_applications =  df.groupby('term')['id'].count()

plt.figure(figsize=(5,5))

plt.pie(
    total_term_applications,
    labels = total_term_applications.index,
    autopct = lambda p: f"{p:.1f}%\n${p*sum(total_term_applications)/100:.1f}M",
    startangle = 90,
    wedgeprops = {'width':0.4}
)

plt.title('No. of Applications by Term')
plt.show()
```



1.0.24 Employee Length by Total Funded Amount

```
emp_funding_thousands = (
    df.groupby('emp_length')['loan_amount'].sum().sort_values()/ 1000)

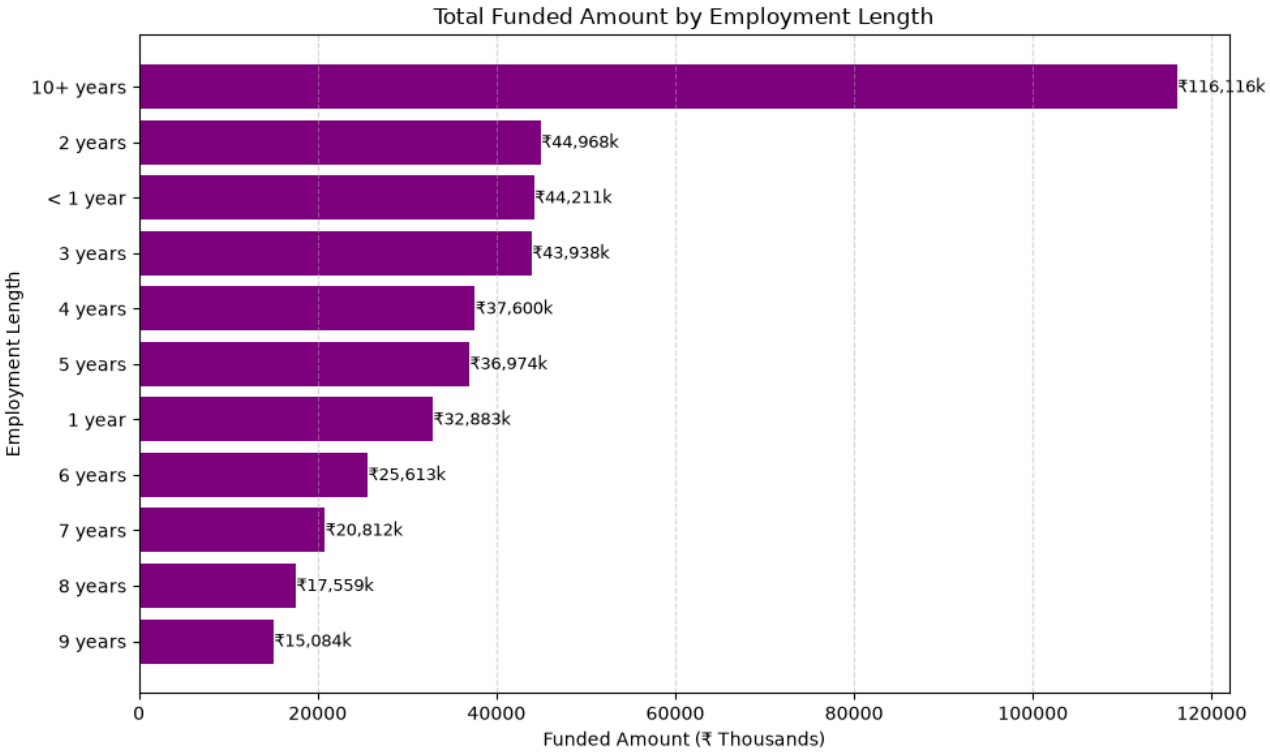
plt.figure(figsize=(10,6))

bars = plt.barh(emp_funding_thousands.index, emp_funding_thousands.values, color =
'purple')

for bar in bars:
    width = bar.get_width()

    plt.text(width + 50, bar.get_y() + bar.get_height()/2, f'₹{width:,.0f}k', va
='center', fontsize = 9 )

plt.title('Total Funded Amount by Employment Length')
plt.xlabel('Funded Amount (₹ Thousands)')
plt.ylabel('Employment Length')
plt.grid(axis = 'x', linestyle = '--', alpha = 0.5)
plt.tight_layout()
plt.show()
```



1.0.25 Employee Length by Total Amount Received

```
emp_received_thousands = (
    df.groupby('emp_length')['total_payment'].sum().sort_values()/ 1000)

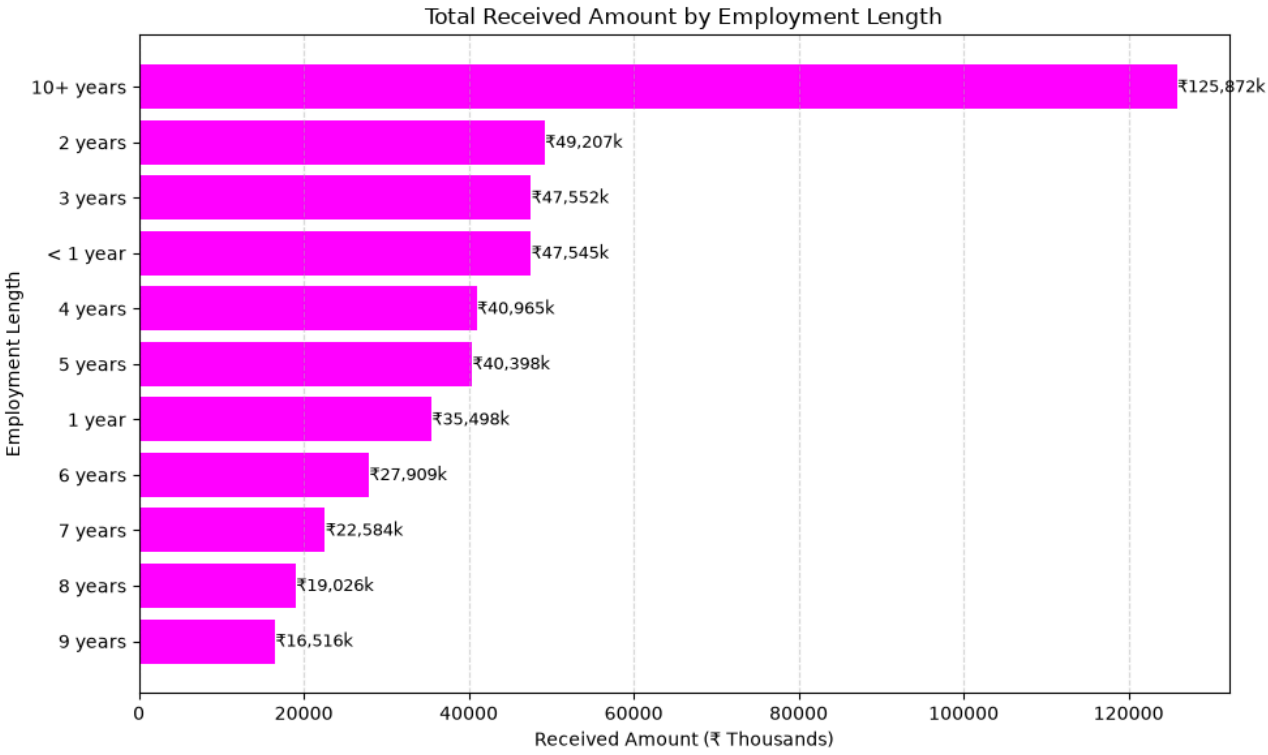
plt.figure(figsize=(10,6))

bars = plt.barh(emp_received_thousands.index, emp_received_thousands.values, color =
'magenta')

for bar in bars:
    width = bar.get_width()

    plt.text(width + 50, bar.get_y() + bar.get_height()/2, f'₹{width:,.0f}k', va
='center', fontsize = 9 )

plt.title('Total Received Amount by Employment Length')
plt.xlabel('Received Amount (₹ Thousands)')
plt.ylabel('Employment Length')
plt.grid(axis = 'x', linestyle = '--', alpha = 0.5)
plt.tight_layout()
plt.show()
```



1.0.26 Employee Length by Total Loan Applications

```
emp_Loan_applications_thousands = (
    df.groupby('emp_length')['id'].count().sort_values())

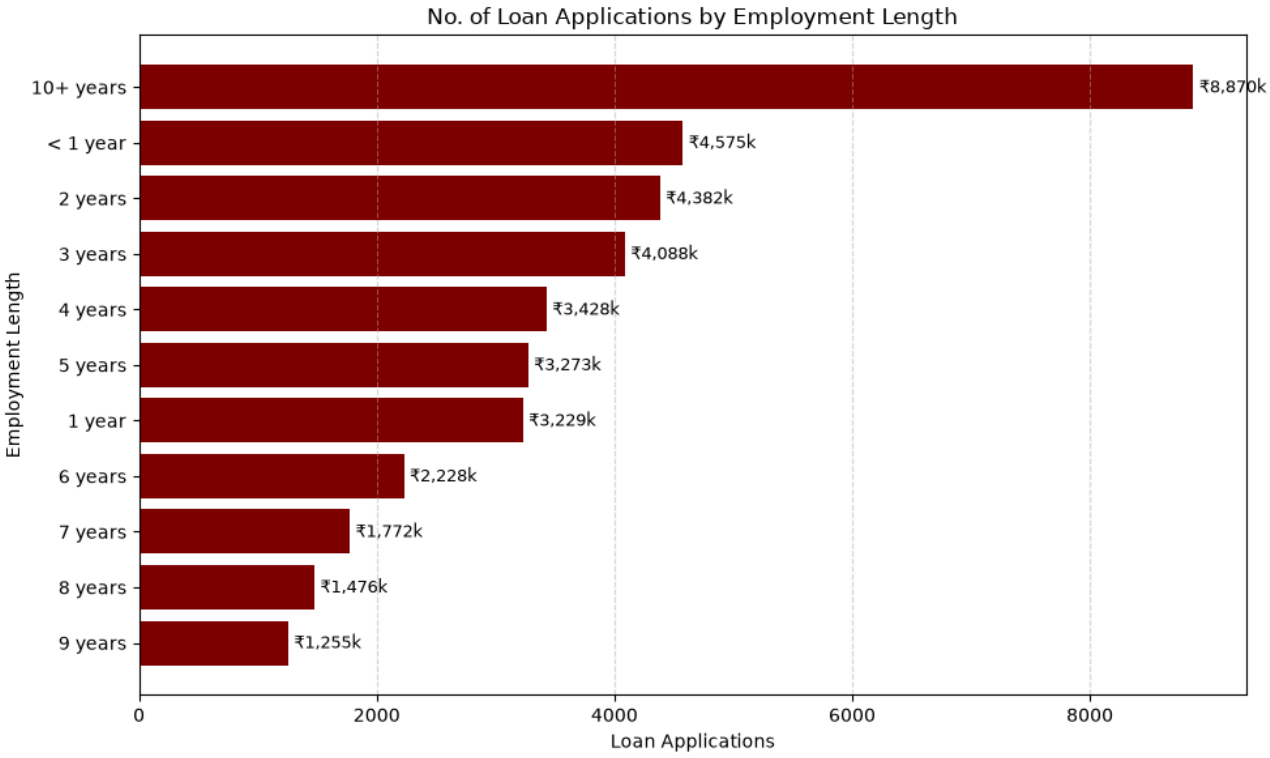
plt.figure(figsize=(10,6))

bars = plt.barh(emp_Loan_applications_thousands.index,
emp_Loan_applications_thousands.values, color = 'maroon')

for bar in bars:
    width = bar.get_width()

    plt.text(width + 50, bar.get_y() + bar.get_height()/2, f'₹{width:,.0f}k', va
='center', fontsize = 9 )

plt.title('No. of Loan Applications by Employment Length')
plt.xlabel('Loan Applications')
plt.ylabel('Employment Length')
plt.grid(axis = 'x', linestyle = '--', alpha = 0.5)
plt.tight_layout()
plt.show()
```



1.0.27 Loan Purpose by Total Funded Amount

```
purpose_funding_millions = (
    df.groupby('purpose')['loan_amount'].sum().sort_values()/ 1000000)

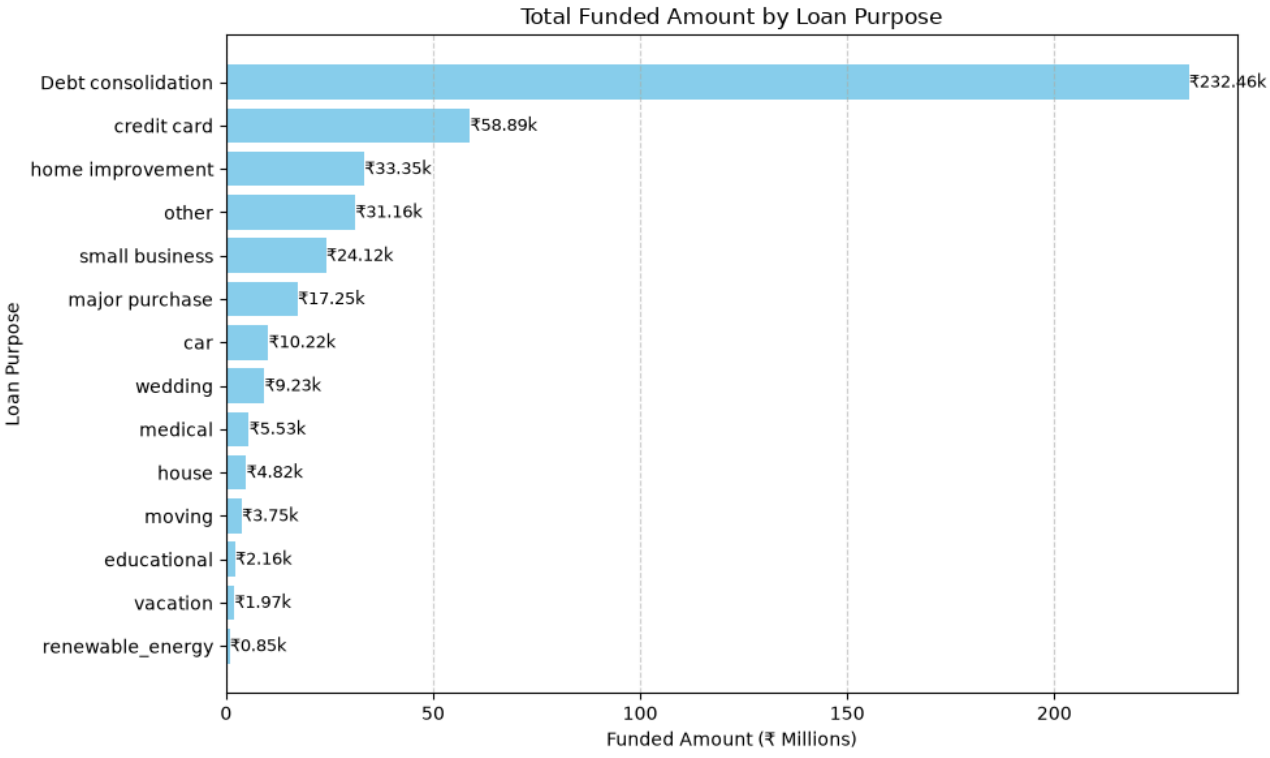
plt.figure(figsize=(10,6))

bars = plt.barh(purpose_funding_millions.index, purpose_funding_millions.values,
color = 'skyblue')

for bar in bars:
    width = bar.get_width()

    plt.text(width + 0.1, bar.get_y() + bar.get_height()/2, f'₹{width:,.2f}k', va
='center', fontsize = 9 )

plt.title('Total Funded Amount by Loan Purpose')
plt.xlabel('Funded Amount (₹ Millions)')
plt.ylabel('Loan Purpose')
plt.grid(axis = 'x', linestyle = '--', alpha = 0.6)
plt.tight_layout()
plt.show()
```



1.0.28 Loan Purpose by Total Amount Received

```
purpose_received_millions = (
    df.groupby('purpose')['total_payment'].sum().sort_values()/ 1000000)

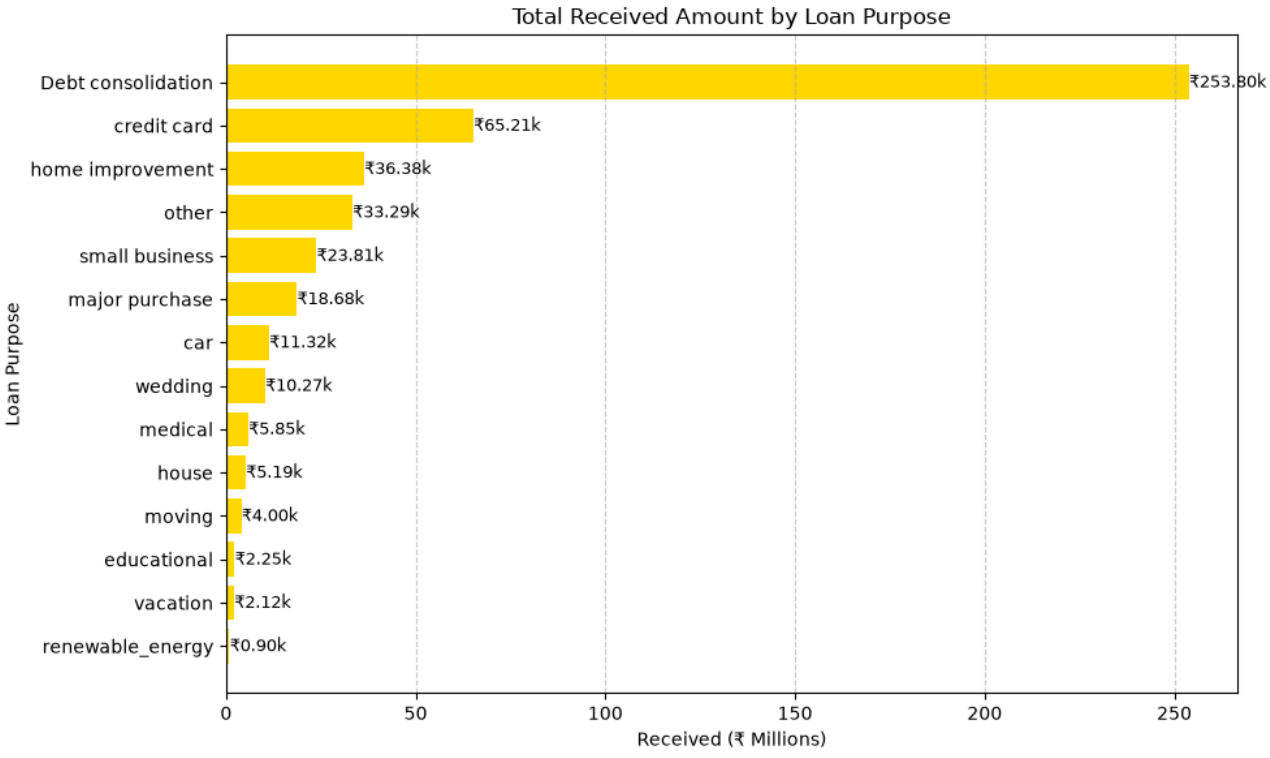
plt.figure(figsize=(10,6))

bars = plt.barh(purpose_received_millions.index, purpose_received_millions.values,
color = 'gold')

for bar in bars:
    width = bar.get_width()

    plt.text(width + 0.1, bar.get_y() + bar.get_height()/2, f'₹{width:,.2f}k', va
='center', fontsize = 9 )

plt.title('Total Received Amount by Loan Purpose')
plt.xlabel('Received (₹ Millions)')
plt.ylabel('Loan Purpose')
plt.grid(axis = 'x', linestyle = '--', alpha = 0.6)
plt.tight_layout()
plt.show()
```



1.0.29 Loan Purpose by Total Loan Applications

```
purpose_loan_applications = (
    df.groupby('purpose')['id'].count().sort_values())

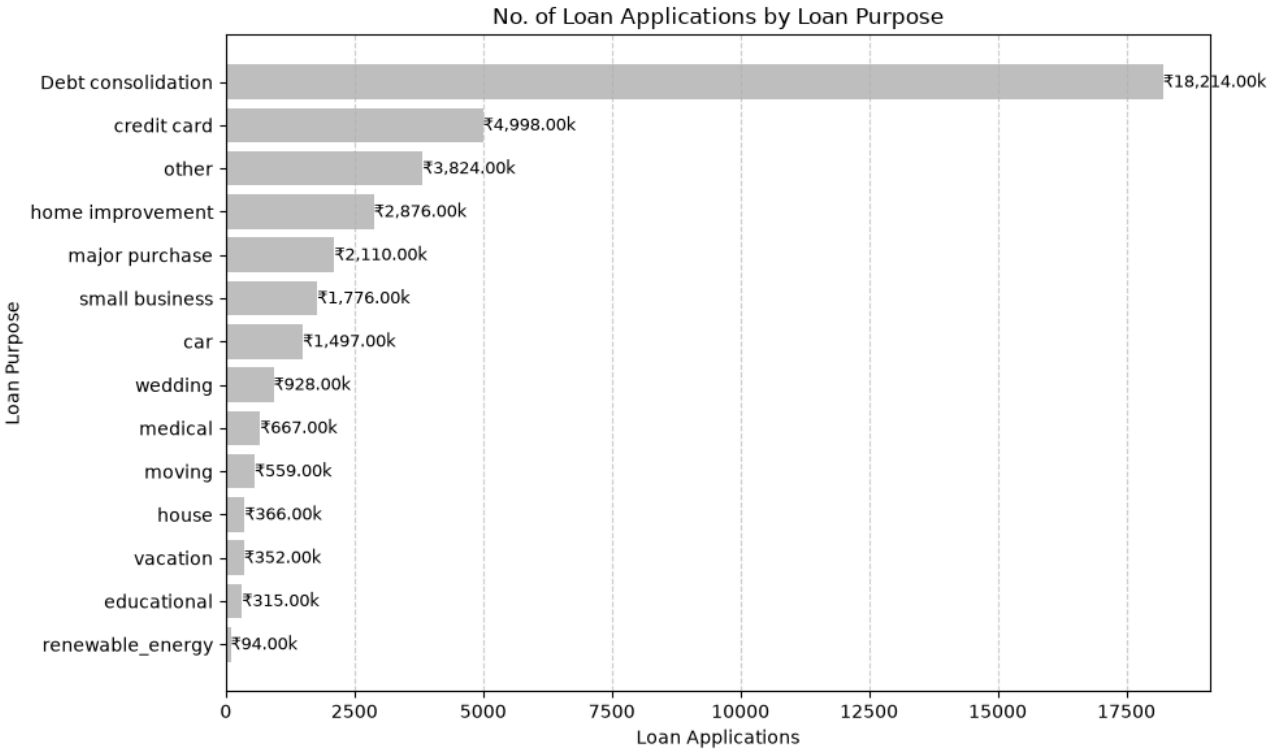
plt.figure(figsize=(10,6))

bars = plt.barh(purpose_loan_applications.index, purpose_loan_applications.values,
color = 'silver')

for bar in bars:
    width = bar.get_width()

    plt.text(width + 0.1, bar.get_y() + bar.get_height()/2, f'₹{width:,.2f}k', va
='center', fontsize = 9 )

plt.title('No. of Loan Applications by Loan Purpose')
plt.xlabel('Loan Applications')
plt.ylabel('Loan Purpose')
plt.grid(axis = 'x', linestyle = '--', alpha = 0.6)
plt.tight_layout()
plt.show()
```



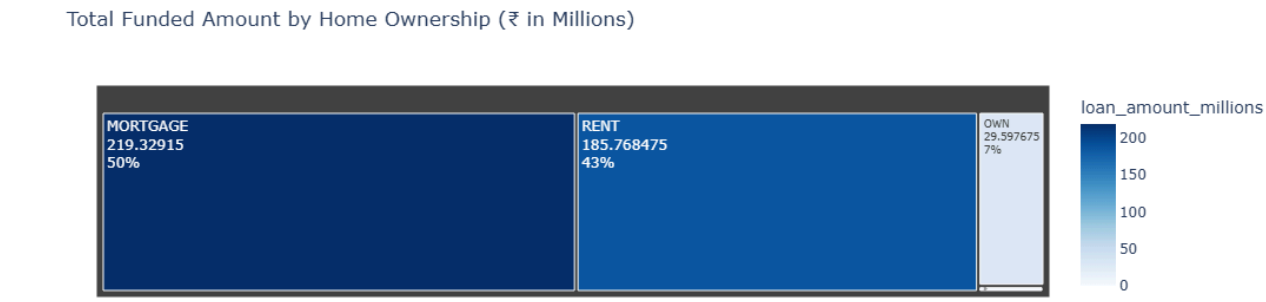
1.0.30 Home Ownership by Total Funded Amount

```
home_funding = df.groupby('home_ownership')['loan_amount'].sum().reset_index()
home_funding['loan_amount_millions'] = home_funding['loan_amount'] / 1_000_000

fig = px.treemap(
    home_funding,
    path = ['home_ownership'],
    values = 'loan_amount_millions',
    color = 'loan_amount_millions',
    color_continuous_scale = 'Blues',
    title = 'Total Funded Amount by Home Ownership (₹ in Millions)'
)

fig.update_traces(
    textinfo = 'label+value+percent root'
)

fig.show()
```



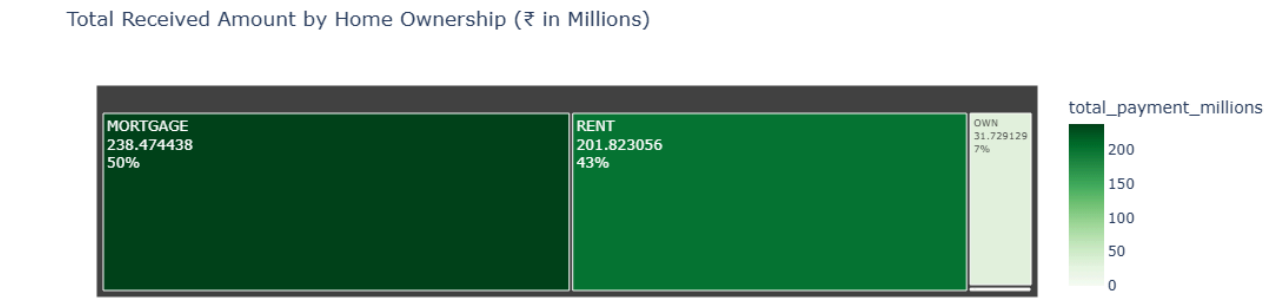
1.0.31 Home Ownership by Total Amount Received

```
home_received = df.groupby('home_ownership')['total_payment'].sum().reset_index()
home_received['total_payment_millions'] = home_received['total_payment'] / 1_000_000

fig = px.treemap(
    home_received,
    path = ['home_ownership'],
    values = 'total_payment_millions',
    color = 'total_payment_millions',
    color_continuous_scale = 'Greens',
    title = 'Total Received Amount by Home Ownership (₹ in Millions)'
)

fig.update_traces(
    textinfo = 'label+value+percent root'
)

fig.show()
```



1.0.32 Home Ownership by Total Loan Applications

```
home_applications = df.groupby('home_ownership')['id'].count().reset_index()
home_applications.rename(columns={'id': 'application_count'}, inplace=True)

fig = px.treemap(
    home_applications,
    path=['home_ownership'],
    values='application_count',
    color='application_count',
    color_continuous_scale='Reds',
    title='Total Loan Applications by Home Ownership'
)

fig.update_traces(
    textinfo = 'label+value+percent root'
)

fig.show()
```

